
The Annotated Transformer

Apr 3, 2018

There is now a new version

(<http://harvardnlp.github.io/annotated-transformer>) of this blog post updated for modern PyTorch.

```
from IPython.display import Image
Image(filename='images/aiayn.png')
```

Attention Is All You Need

Ashish Vaswani*
Google Brain
avaswani@google.com

Noam Shazeer*
Google Brain
noam@google.com

Niki Parmar*
Google Research
nikip@google.com

Jakob Uszkoreit*
Google Research
usz@google.com

Llion Jones*
Google Research
llion@google.com

Aidan N. Gomez* †
University of Toronto
aidan@cs.toronto.edu

Lukasz Kaiser*
Google Brain
lukaszkaier@google.com

Illia Polosukhin* ‡
illia.polosukhin@gmail.com

Abstract

The dominant sequence transduction models are based on complex recurrent or convolutional neural networks that include an encoder and a decoder. The best performing models also connect the encoder and decoder through an attention mechanism. We propose a new simple network architecture, the Transformer, based solely on attention mechanisms, dispensing with recurrence and convolutions entirely. Experiments on two machine translation tasks show these models to be superior in quality while being more parallelizable and requiring significantly less time to train. Our model achieves 28.4 BLEU on the WMT 2014 English-to-German translation task, improving over the existing best results, including ensembles, by over 2 BLEU. On the WMT 2014 English-to-French translation task, our model establishes a new single-model state-of-the-art BLEU score of 41.8 after training for 3.5 days on eight GPUs, a small fraction of the training costs of the best models from the literature. We show that the Transformer generalizes well to other tasks by applying it successfully to English constituency parsing both with large and limited training data.

The Transformer from “Attention is All You Need” (<https://arxiv.org/abs/1706.03762>) has been on a lot of people’s minds over the last year. Besides producing major improvements in translation quality, it provides a new architecture for many other NLP tasks. The paper itself is very clearly written, but the conventional wisdom has been that it is quite difficult to implement correctly.

In this post I present an “annotated” version of the paper in the form of a line-by-line implementation. I have reordered and deleted some sections from the original paper and added comments throughout. This document itself is a working notebook, and should be a

completely usable implementation. In total there are 400 lines of library code which can process 27,000 tokens per second on 4 GPUs.

To follow along you will first need to install PyTorch (<http://pytorch.org/>). The complete notebook is also available on github (<https://github.com/harvardnlp/annotated-transformer>) or on Google Colab (<https://drive.google.com/file/d/1xQXSv6mtAOLXxEMi8RvaW8TW-7bvYBDF/view?usp=sharing>) with free GPUs.

Note this is merely a starting point for researchers and interested developers. The code here is based heavily on our OpenNMT (<http://opennmt.net>) packages. (If helpful feel free to cite.) For other full-service implementations of the model check-out Tensor2Tensor (<https://github.com/tensorflow/tensor2tensor>) (tensorflow) and Sockeye (<https://github.com/aws-labs/sockeye>) (mxnet).

- Alexander Rush (@harvardnlp (<https://twitter.com/harvardnlp>) or rsrush@seas.harvard.edu), with help from Vincent Nguyen and Guillaume Klein

Prelims

```
# !pip install http://download.pytorch.org/whl/cu80/torch-0.3.0.post4-cp36-cp36m-linux_x86_64.whl (http://download.pytorch.org/whl/cu80/torch-0.3.0.post4-cp36-cp36m-linux_x86_64.whl) numpy matplotlib spacy torchtext seaborn
```

```
import numpy as np
import torch
import torch.nn as nn
import torch.nn.functional as F
import math, copy, time
from torch.autograd import Variable
import matplotlib.pyplot as plt
import seaborn
seaborn.set_context(context="talk")
%matplotlib inline
```

Table of Contents

- Prelims
- Background
- Model Architecture
 - Encoder and Decoder Stacks
 - Encoder
 - Decoder
 - Attention
 - Applications of Attention in our Model
 - Position-wise Feed-Forward Networks
 - Embeddings and Softmax
 - Positional Encoding
 - Full Model
- Training
 - Batches and Masking
 - Training Loop
 - Training Data and Batching
 - Hardware and Schedule
 - Optimizer
 - Regularization
 - Label Smoothing
- A First Example
 - Synthetic Data
 - Loss Computation

- Greedy Decoding
- A Real World Example
 - Data Loading
 - Iterators
 - Multi-GPU Training
 - Training the System
- Additional Components: BPE, Search, Averaging
- Results
 - Attention Visualization
- Conclusion

My comments are blockquoted. The main text is all from the paper itself.

Background

The goal of reducing sequential computation also forms the foundation of the Extended Neural GPU, ByteNet and ConvS2S, all of which use convolutional neural networks as basic building block, computing hidden representations in parallel for all input and output positions. In these models, the number of operations required to relate signals from two arbitrary input or output positions grows in the distance between positions, linearly for ConvS2S and logarithmically for ByteNet. This makes it more difficult to learn dependencies between distant positions. In the Transformer this is reduced to a constant number of operations, albeit at the cost of reduced effective resolution due to averaging attention-weighted positions, an effect we counteract with Multi-Head Attention.

Self-attention, sometimes called intra-attention is an attention mechanism relating different positions of a single sequence in order to compute a representation of the sequence. Self-attention has been used successfully in a variety of tasks including reading comprehension, abstractive summarization, textual entailment and learning task-independent sentence representations. End- to-end

memory networks are based on a recurrent attention mechanism instead of sequence-aligned recurrence and have been shown to perform well on simple- language question answering and language modeling tasks.

To the best of our knowledge, however, the Transformer is the first transduction model relying entirely on self-attention to compute representations of its input and output without using sequence aligned RNNs or convolution.

Model Architecture

Most competitive neural sequence transduction models have an encoder-decoder structure (cite) (<https://arxiv.org/abs/1409.0473>). Here, the encoder maps an input sequence of symbol representations $(x_1, \dots, x_n)$ to a sequence of continuous representations $\mathbf{z} = (z_1, \dots, z_n)$. Given $\mathbf{z}$, the decoder then generates an output sequence $(y_1, \dots, y_m)$ of symbols one element at a time. At each step the model is auto-regressive (cite) (<https://arxiv.org/abs/1308.0850>), consuming the previously generated symbols as additional input when generating the next.

```

class EncoderDecoder(nn.Module):
    """
    A standard Encoder-Decoder architecture. Base for
    this and many other models.
    """
    def __init__(self, encoder, decoder, src_embed,
tgt_embed, generator):
        super(EncoderDecoder, self).__init__()
        self.encoder = encoder
        self.decoder = decoder
        self.src_embed = src_embed
        self.tgt_embed = tgt_embed
        self.generator = generator

    def forward(self, src, tgt, src_mask, tgt_mask):
        "Take in and process masked src and target sequences."
        return self.decode(self.encode(src, src_mask),
                                tgt, tgt_mask)

    def encode(self, src, src_mask):
        return self.encoder(self.src_embed(src), src_mask)

    def decode(self, memory, src_mask, tgt, tgt_mask):
        return self.decoder(self.tgt_embed(tgt), memory, src_mask, tgt_mask)

```

```

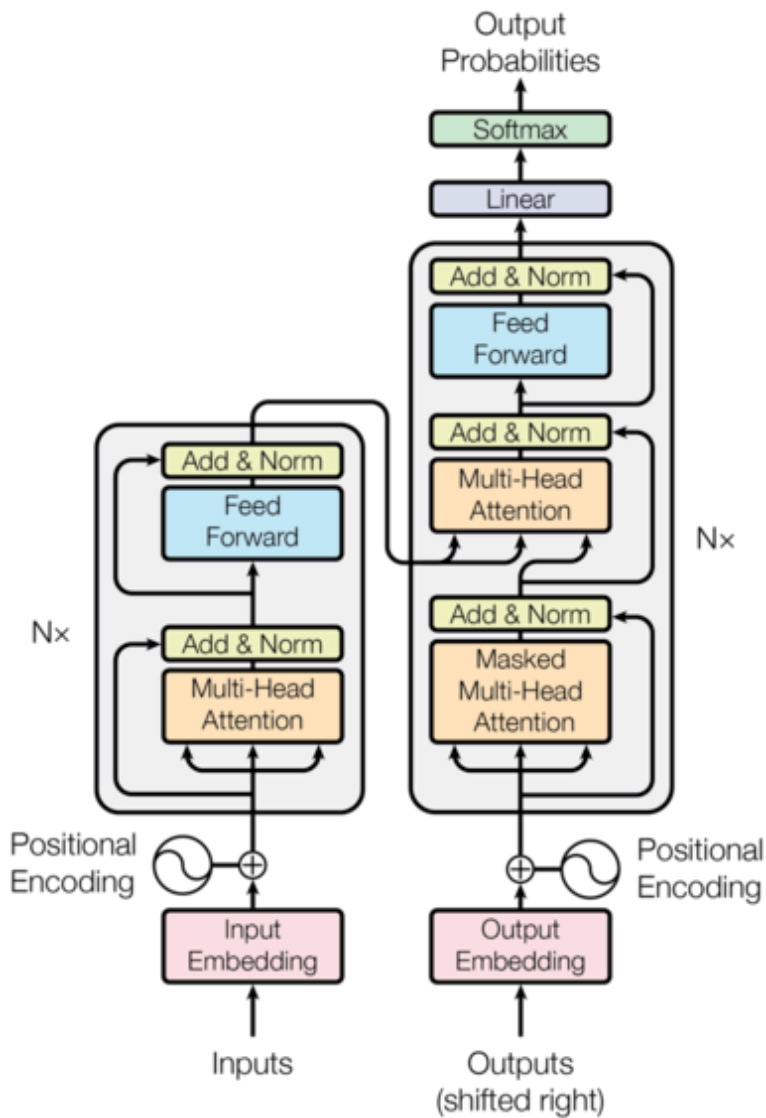
class Generator(nn.Module):
    "Define standard linear + softmax generation step."
    def __init__(self, d_model, vocab):
        super(Generator, self).__init__()
        self.proj = nn.Linear(d_model, vocab)

    def forward(self, x):
        return F.log_softmax(self.proj(x), dim=-1)

```

The Transformer follows this overall architecture using stacked self-attention and point-wise, fully connected layers for both the encoder and decoder, shown in the left and right halves of Figure 1, respectively.

```
Image(filename='images/ModalNet-21.png')
```



Encoder and Decoder Stacks

Encoder

The encoder is composed of a stack of $N = 6$ identical layers.

```
def clones(module, N):  
    "Produce N identical layers."  
    return nn.ModuleList([copy.deepcopy(module) for  
_ in range(N)])
```

```
class Encoder(nn.Module):  
    "Core encoder is a stack of N layers"  
    def __init__(self, layer, N):  
        super(Encoder, self).__init__()  
        self.layers = clones(layer, N)  
        self.norm = LayerNorm(layer.size)  
  
    def forward(self, x, mask):  
        "Pass the input (and mask) through each layer  
in turn."  
        for layer in self.layers:  
            x = layer(x, mask)  
        return self.norm(x)
```

We employ a residual connection (cite
(<https://arxiv.org/abs/1512.03385>) around each of the two sub-
layers, followed by layer normalization (cite
(<https://arxiv.org/abs/1607.06450>)).

```

class LayerNorm(nn.Module):
    "Construct a layernorm module (See citation for
    details)."  

    def __init__(self, features, eps=1e-6):  

        super(LayerNorm, self).__init__()  

        self.a_2 = nn.Parameter(torch.ones(feature  
s))  

        self.b_2 = nn.Parameter(torch.zeros(feature  
s))  

        self.eps = eps  
  

    def forward(self, x):  

        mean = x.mean(-1, keepdim=True)  

        std = x.std(-1, keepdim=True)  

        return self.a_2 * (x - mean) / (std + self.e  
ps) + self.b_2

```

That is, the output of each sub-layer is

$\text{LayerNorm}(x + \text{Sublayer}(x))$, where $\text{Sublayer}(x)$ is the function implemented by the sub-layer itself. We apply dropout (cite) (<http://jmlr.org/papers/v15/srivastava14a.html>) to the output of each sub-layer, before it is added to the sub-layer input and normalized.

To facilitate these residual connections, all sub-layers in the model, as well as the embedding layers, produce outputs of dimension $d_{\text{model}} = 512$.

```

class SublayerConnection(nn.Module):
    """
    A residual connection followed by a layer norm.
    Note for code simplicity the norm is first as op
    posed to last.
    """
    def __init__(self, size, dropout):
        super(SublayerConnection, self).__init__()
        self.norm = LayerNorm(size)
        self.dropout = nn.Dropout(dropout)

    def forward(self, x, sublayer):
        "Apply residual connection to any sublayer w
        ith the same size."
        return x + self.dropout(sublayer(self.norm
        (x)))

```

Each layer has two sub-layers. The first is a multi-head self-attention mechanism, and the second is a simple, position-wise fully connected feed- forward network.

```

class EncoderLayer(nn.Module):
    "Encoder is made up of self-attn and feed forward
    d (defined below)"
    def __init__(self, size, self_attn, feed_forward, dropout):
        super(EncoderLayer, self).__init__()
        self.self_attn = self_attn
        self.feed_forward = feed_forward
        self.sublayer = clones(SublayerConnection(size, dropout), 2)
        self.size = size

    def forward(self, x, mask):
        "Follow Figure 1 (left) for connections."
        x = self.sublayer[0](x, lambda x: self.self_attn(x, x, x, mask))
        return self.sublayer[1](x, self.feed_forward)

```

Decoder

The decoder is also composed of a stack of $N = 6$ identical layers.

```

class Decoder(nn.Module):
    "Generic N layer decoder with masking."
    def __init__(self, layer, N):
        super(Decoder, self).__init__()
        self.layers = clones(layer, N)
        self.norm = LayerNorm(layer.size)

    def forward(self, x, memory, src_mask, tgt_mask):
        for layer in self.layers:
            x = layer(x, memory, src_mask, tgt_mask)
        return self.norm(x)

```

In addition to the two sub-layers in each encoder layer, the decoder inserts a third sub-layer, which performs multi-head attention over the output of the encoder stack. Similar to the encoder, we employ residual connections around each of the sub-layers, followed by layer normalization.

```
class DecoderLayer(nn.Module):
    "Decoder is made of self-attn, src-attn, and feed forward (defined below)"
    def __init__(self, size, self_attn, src_attn, feed_forward, dropout):
        super(DecoderLayer, self).__init__()
        self.size = size
        self.self_attn = self_attn
        self.src_attn = src_attn
        self.feed_forward = feed_forward
        self.sublayer = clones(SublayerConnection(size, dropout), 3)

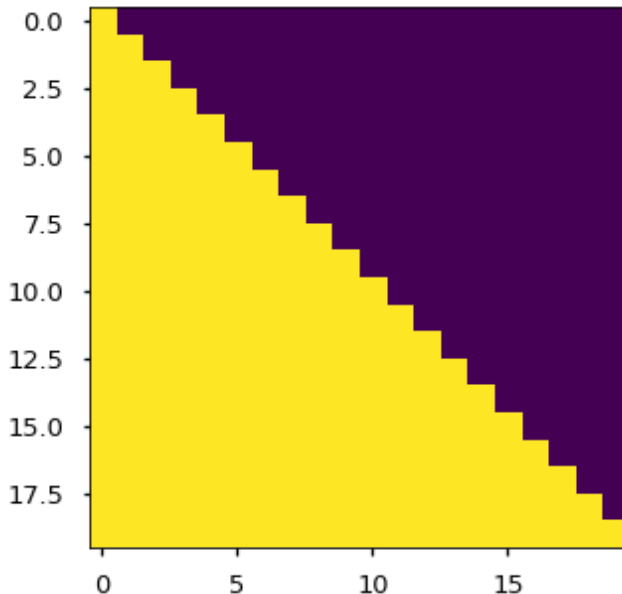
    def forward(self, x, memory, src_mask, tgt_mask):
        "Follow Figure 1 (right) for connections."
        m = memory
        x = self.sublayer[0](x, lambda x: self.self_attn(x, x, x, tgt_mask))
        x = self.sublayer[1](x, lambda x: self.src_attn(x, m, m, src_mask))
        return self.sublayer[2](x, self.feed_forward)
```

We also modify the self-attention sub-layer in the decoder stack to prevent positions from attending to subsequent positions. This masking, combined with fact that the output embeddings are offset by one position, ensures that the predictions for position i can depend only on the known outputs at positions less than i .

```
def subsequent_mask(size):
    "Mask out subsequent positions."
    attn_shape = (1, size, size)
    subsequent_mask = np.triu(np.ones(attn_shape), k
=1).astype('uint8')
    return torch.from_numpy(subsequent_mask) == 0
```

Below the attention mask shows the position each tgt word (row) is allowed to look at (column). Words are blocked for attending to future words during training.

```
plt.figure(figsize=(5,5))
plt.imshow(subsequent_mask(20)[0])
None
```

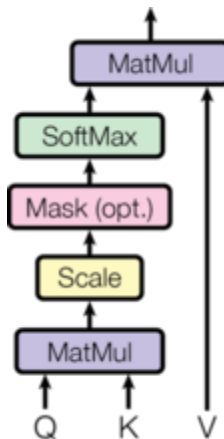


Attention

An attention function can be described as mapping a query and a set of key-value pairs to an output, where the query, keys, values, and output are all vectors. The output is computed as a weighted sum of the values, where the weight assigned to each value is computed by a compatibility function of the query with the corresponding key.

We call our particular attention “Scaled Dot-Product Attention”. The input consists of queries and keys of dimension d_k , and values of dimension d_v . We compute the dot products of the query with all keys, divide each by $\sqrt{d_k}$, and apply a softmax function to obtain the weights on the values.

```
Image(filename='images/ModalNet-19.png')
```



In practice, we compute the attention function on a set of queries simultaneously, packed together into a matrix Q . The keys and values are also packed together into matrices K and V . We compute the matrix of outputs as:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

```

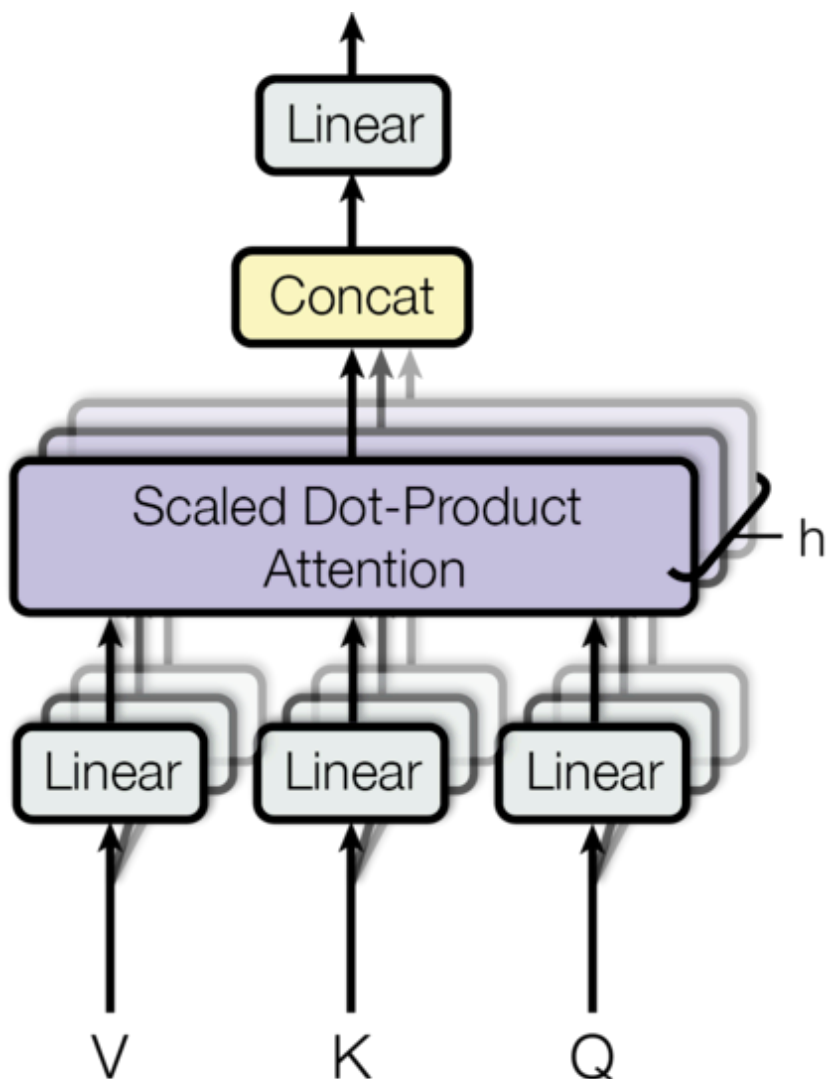
def attention(query, key, value, mask=None, dropout=
None):
    "Compute 'Scaled Dot Product Attention'"
    d_k = query.size(-1)
    scores = torch.matmul(query, key.transpose(-2, -
1)) \
        / math.sqrt(d_k)
    if mask is not None:
        scores = scores.masked_fill(mask == 0, -1e9)
    p_attn = F.softmax(scores, dim = -1)
    if dropout is not None:
        p_attn = dropout(p_attn)
    return torch.matmul(p_attn, value), p_attn

```

The two most commonly used attention functions are additive attention (cite) (<https://arxiv.org/abs/1409.0473>), and dot-product (multiplicative) attention. Dot-product attention is identical to our algorithm, except for the scaling factor of $\frac{1}{\sqrt{d_k}}$. Additive attention computes the compatibility function using a feed-forward network with a single hidden layer. While the two are similar in theoretical complexity, dot-product attention is much faster and more space-efficient in practice, since it can be implemented using highly optimized matrix multiplication code.

While for small values of d_k the two mechanisms perform similarly, additive attention outperforms dot product attention without scaling for larger values of d_k (cite) (<https://arxiv.org/abs/1703.03906>). We suspect that for large values of d_k , the dot products grow large in magnitude, pushing the softmax function into regions where it has extremely small gradients (To illustrate why the dot products get large, assume that the components of q and k are independent random variables with mean 0 and variance 1. Then their dot product, $q \cdot k = \sum_{i=1}^{d_k} q_i k_i$, has mean 0 and variance d_k). To counteract this effect, we scale the dot products by $\frac{1}{\sqrt{d_k}}$.

```
Image(filename='images/ModalNet-20.png')
```



Multi-head attention allows the model to jointly attend to information from different representation subspaces at different positions. With a single attention head, averaging inhibits this.

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W^O$$

$$\text{where } \text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$$

Where the projections are parameter matrices $W_i^Q \in \mathbb{R}^{d_{\text{model}} \times d_k}$, $W_i^K \in \mathbb{R}^{d_{\text{model}} \times d_k}$, $W_i^V \in \mathbb{R}^{d_{\text{model}} \times d_v}$ and $W^O \in \mathbb{R}^{hd_v \times d_{\text{model}}}$. In this work we employ $h = 8$ parallel attention layers, or heads. For each of these we use $d_k = d_v = d_{\text{model}}/h = 64$. Due to the reduced dimension of each head, the total computational cost is similar to that of single-head attention with full dimensionality.

```

class MultiHeadedAttention(nn.Module):
    def __init__(self, h, d_model, dropout=0.1):
        "Take in model size and number of heads."
        super(MultiHeadedAttention, self).__init__()
        assert d_model % h == 0
        # We assume d_v always equals d_k
        self.d_k = d_model // h
        self.h = h
        self.linears = clones(nn.Linear(d_model, d_model), 4)

        self.attn = None
        self.dropout = nn.Dropout(p=dropout)

    def forward(self, query, key, value, mask=None):
        "Implements Figure 2"
        if mask is not None:
            # Same mask applied to all h heads.
            mask = mask.unsqueeze(1)
            nbatches = query.size(0)

            # 1) Do all the linear projections in batch
from d_model => h x d_k
            query, key, value = \
                [l(x).view(nbatches, -1, self.h, self.d_k).transpose(1, 2)
                 for l, x in zip(self.linears, (query, key, value)))]

            # 2) Apply attention on all the projected vectors in batch.
            x, self.attn = attention(query, key, value, mask=mask,
                                     dropout=self.dropout)

            # 3) "Concat" using a view and apply a final linear.
            x = x.transpose(1, 2).contiguous() \

```

```
        .view(nbatches, -1, self.h * self.d_k)
    return self.linears[-1](x)
```

Applications of Attention in our Model

The Transformer uses multi-head attention in three different ways: 1) In “encoder-decoder attention” layers, the queries come from the previous decoder layer, and the memory keys and values come from the output of the encoder. This allows every position in the decoder to attend over all positions in the input sequence. This mimics the typical encoder-decoder attention mechanisms in sequence-to-sequence models such as (cite) (<https://arxiv.org/abs/1609.08144>).

2) The encoder contains self-attention layers. In a self-attention layer all of the keys, values and queries come from the same place, in this case, the output of the previous layer in the encoder. Each position in the encoder can attend to all positions in the previous layer of the encoder.

3) Similarly, self-attention layers in the decoder allow each position in the decoder to attend to all positions in the decoder up to and including that position. We need to prevent leftward information flow in the decoder to preserve the auto-regressive property. We implement this inside of scaled dot-product attention by masking out (setting to $-\infty$) all values in the input of the softmax which correspond to illegal connections.

Position-wise Feed-Forward Networks

In addition to attention sub-layers, each of the layers in our encoder and decoder contains a fully connected feed-forward network, which is applied to each position separately and identically. This

consists of two linear transformations with a ReLU activation in between.

$$\text{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2$$

While the linear transformations are the same across different positions, they use different parameters from layer to layer. Another way of describing this is as two convolutions with kernel size 1. The dimensionality of input and output is $d_{\text{model}} = 512$, and the inner-layer has dimensionality $d_{\text{ff}} = 2048$.

```
class PositionwiseFeedForward(nn.Module):  
    "Implements FFN equation."  
    def __init__(self, d_model, d_ff, dropout=0.1):  
        super(PositionwiseFeedForward, self).__init__  
_()  
        self.w_1 = nn.Linear(d_model, d_ff)  
        self.w_2 = nn.Linear(d_ff, d_model)  
        self.dropout = nn.Dropout(dropout)  
  
    def forward(self, x):  
        return self.w_2(self.dropout(F.relu(self.w_1  
(x))))
```

Embeddings and Softmax

Similarly to other sequence transduction models, we use learned embeddings to convert the input tokens and output tokens to vectors of dimension d_{model} . We also use the usual learned linear transformation and softmax function to convert the decoder output to predicted next-token probabilities. In our model, we share the same weight matrix between the two embedding layers and the pre-softmax linear transformation, similar to (cite) (<https://arxiv.org/abs/1608.05859>). In the embedding layers, we multiply those weights by $\sqrt{d_{\text{model}}}$.

```

class Embeddings(nn.Module):
    def __init__(self, d_model, vocab):
        super(Embeddings, self).__init__()
        self.lut = nn.Embedding(vocab, d_model)
        self.d_model = d_model

    def forward(self, x):
        return self.lut(x) * math.sqrt(self.d_model)

```

Positional Encoding

Since our model contains no recurrence and no convolution, in order for the model to make use of the order of the sequence, we must inject some information about the relative or absolute position of the tokens in the sequence. To this end, we add “positional encodings” to the input embeddings at the bottoms of the encoder and decoder stacks. The positional encodings have the same dimension d_{model} as the embeddings, so that the two can be summed. There are many choices of positional encodings, learned and fixed (cite) (<https://arxiv.org/pdf/1705.03122.pdf>).

In this work, we use sine and cosine functions of different frequencies: $PE_{(pos, 2i)} = \sin(pos/10000^{2i/d_{\text{model}}})$

$PE_{(pos, 2i+1)} = \cos(pos/10000^{2i/d_{\text{model}}})$ where pos is the position and i is the dimension. That is, each dimension of the positional encoding corresponds to a sinusoid. The wavelengths form a geometric progression from 2π to $10000 \cdot 2\pi$. We chose this function because we hypothesized it would allow the model to easily learn to attend by relative positions, since for any fixed offset k , PE_{pos+k} can be represented as a linear function of PE_{pos} .

In addition, we apply dropout to the sums of the embeddings and the positional encodings in both the encoder and decoder stacks. For the base model, we use a rate of $P_{\text{drop}} = 0.1$.

```

class PositionalEncoding(nn.Module):
    "Implement the PE function."
    def __init__(self, d_model, dropout, max_len=5000):
        super(PositionalEncoding, self).__init__()
        self.dropout = nn.Dropout(p=dropout)

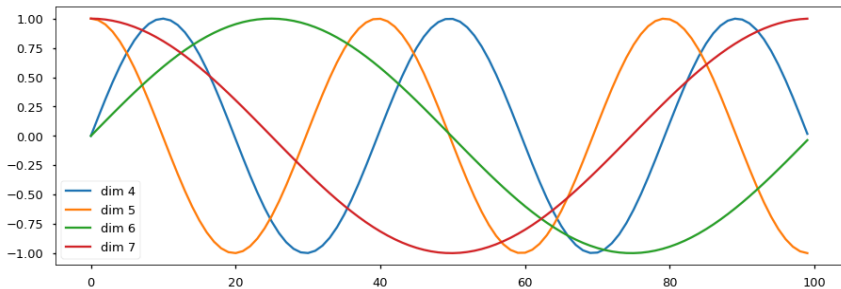
        # Compute the positional encodings once in log space.
        pe = torch.zeros(max_len, d_model)
        position = torch.arange(0, max_len).unsqueeze(1)
        div_term = torch.exp(torch.arange(0, d_model, 2) *
                                $-(\text{math.log}(10000.0) / d\_model)$ )
        pe[:, 0::2] = torch.sin(position * div_term)
        pe[:, 1::2] = torch.cos(position * div_term)
        pe = pe.unsqueeze(0)
        self.register_buffer('pe', pe)

    def forward(self, x):
        x = x + Variable(self.pe[:, :x.size(1)],
                        requires_grad=False)
        return self.dropout(x)

```

Below the positional encoding will add in a sine wave based on position. The frequency and offset of the wave is different for each dimension.

```
plt.figure(figsize=(15, 5))
pe = PositionalEncoding(20, 0)
y = pe.forward(Variable(torch.zeros(1, 100, 20)))
plt.plot(np.arange(100), y[0, :, 4:8].data.numpy())
plt.legend(["dim %d"%p for p in [4,5,6,7]])
None
```



We also experimented with using learned positional embeddings (cite) (<https://arxiv.org/pdf/1705.03122.pdf>) instead, and found that the two versions produced nearly identical results. We chose the sinusoidal version because it may allow the model to extrapolate to sequence lengths longer than the ones encountered during training.

Full Model

Here we define a function that takes in hyperparameters and produces a full model.

```

def make_model(src_vocab, tgt_vocab, N=6,
               d_model=512, d_ff=2048, h=8, dropout=
0.1):
    "Helper: Construct a model from hyperparameter
s."
    c = copy.deepcopy
    attn = MultiHeadedAttention(h, d_model)
    ff = PositionwiseFeedForward(d_model, d_ff, drop
out)
    position = PositionalEncoding(d_model, dropout)
    model = EncoderDecoder(
        Encoder(EncoderLayer(d_model, c(attn), c(f
f), dropout), N),
        Decoder(DecoderLayer(d_model, c(attn), c(att
n),
                    c(ff), dropout), N),
        nn.Sequential(Embeddings(d_model, src_voca
b), c(position)),
        nn.Sequential(Embeddings(d_model, tgt_voca
b), c(position)),
        Generator(d_model, tgt_vocab))

    # This was important from their code.
    # Initialize parameters with Glorot / fan_avg.
    for p in model.parameters():
        if p.dim() > 1:
            nn.init.xavier_uniform(p)
    return model

```

```

# Small example model.
tmp_model = make_model(10, 10, 2)
None

```

Training

This section describes the training regime for our models.

We stop for a quick interlude to introduce some of the tools needed to train a standard encoder decoder model. First we define a batch object that holds the src and target sentences for training, as well as constructing the masks.

Batches and Masking

```
class Batch:
    "Object for holding a batch of data with mask during training."
    def __init__(self, src, trg=None, pad=0):
        self.src = src
        self.src_mask = (src != pad).unsqueeze(-2)
        if trg is not None:
            self.trg = trg[:, :-1]
            self.trg_y = trg[:, 1:]
            self.trg_mask = \
                self.make_std_mask(self.trg, pad)
            self.ntokens = (self.trg_y != pad).data.sum()

    @staticmethod
    def make_std_mask(tgt, pad):
        "Create a mask to hide padding and future words."
        tgt_mask = (tgt != pad).unsqueeze(-2)
        tgt_mask = tgt_mask & Variable(
            subsequent_mask(tgt.size(-1)).type_as(tgt_mask.data))
        return tgt_mask
```

Next we create a generic training and scoring function to keep track of loss. We pass in a generic loss compute function that also handles parameter updates.

Training Loop

```
def run_epoch(data_iter, model, loss_compute):
    "Standard Training and Logging Function"
    start = time.time()
    total_tokens = 0
    total_loss = 0
    tokens = 0
    for i, batch in enumerate(data_iter):
        out = model.forward(batch.src, batch.trg,
                             batch.src_mask, batch.trg_mask)
        loss = loss_compute(out, batch.trg_y, batch.ntokens)
        total_loss += loss
        total_tokens += batch.ntokens
        tokens += batch.ntokens
        if i % 50 == 1:
            elapsed = time.time() - start
            print("Epoch Step: %d Loss: %f Tokens per Sec: %f" %
                  (i, loss / batch.ntokens, tokens / elapsed))
            start = time.time()
            tokens = 0
    return total_loss / total_tokens
```

Training Data and Batching

We trained on the standard WMT 2014 English-German dataset consisting of about 4.5 million sentence pairs. Sentences were encoded using byte-pair encoding, which has a shared source-target vocabulary of about 37000 tokens. For English- French, we used the significantly larger WMT 2014 English-French dataset consisting of 36M sentences and split tokens into a 32000 word-piece vocabulary.

Sentence pairs were batched together by approximate sequence length. Each training batch contained a set of sentence pairs containing approximately 25000 source tokens and 25000 target tokens.

We will use torch text for batching. This is discussed in more detail below. Here we create batches in a torchtext function that ensures our batch size padded to the maximum batchsize does not surpass a threshold (25000 if we have 8 gpus).

```

global max_src_in_batch, max_tgt_in_batch
def batch_size_fn(new, count, sofar):
    "Keep augmenting batch and calculate total number of tokens + padding."
    global max_src_in_batch, max_tgt_in_batch
    if count == 1:
        max_src_in_batch = 0
        max_tgt_in_batch = 0
    max_src_in_batch = max(max_src_in_batch, len(new.src))
    max_tgt_in_batch = max(max_tgt_in_batch, len(new.trg) + 2)
    src_elements = count * max_src_in_batch
    tgt_elements = count * max_tgt_in_batch
    return max(src_elements, tgt_elements)

```

Hardware and Schedule

We trained our models on one machine with 8 NVIDIA P100 GPUs. For our base models using the hyperparameters described throughout the paper, each training step took about 0.4 seconds. We trained the base models for a total of 100,000 steps or 12 hours. For our big models, step time was 1.0 seconds. The big models were trained for 300,000 steps (3.5 days).

Optimizer

We used the Adam optimizer (cite) (<https://arxiv.org/abs/1412.6980>) with $\beta_1 = 0.9$, $\beta_2 = 0.98$ and $\epsilon = 10^{-9}$. We varied the learning rate over the course of training, according to the formula:

$$lrate = d_{\text{model}}^{-0.5} \cdot \min(step_num^{-0.5}, step_num \cdot warmup_steps^{-1.5})$$

This corresponds to increasing the learning rate linearly for the first $warmup_steps$ training steps, and decreasing it thereafter proportionally to the inverse square root of the step number. We used $warmup_steps = 4000$.

Note: This part is very important. Need to train with this setup of the model.

```

class NoamOpt:
    "Optim wrapper that implements rate."
    def __init__(self, model_size, factor, warmup, optimizer):
        self.optimizer = optimizer
        self._step = 0
        self.warmup = warmup
        self.factor = factor
        self.model_size = model_size
        self._rate = 0

    def step(self):
        "Update parameters and rate"
        self._step += 1
        rate = self.rate()
        for p in self.optimizer.param_groups:
            p['lr'] = rate
        self._rate = rate
        self.optimizer.step()

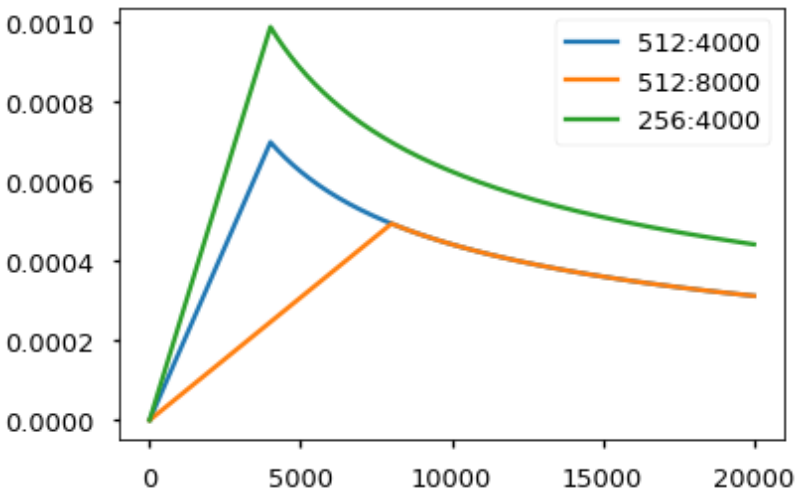
    def rate(self, step = None):
        "Implement `lr` above"
        if step is None:
            step = self._step
        return self.factor * \
            (self.model_size ** (-0.5) *
             min(step ** (-0.5), step * self.warmup *
                * (-1.5)))

    def get_std_opt(model):
        return NoamOpt(model.src_embed[0].d_model, 2, 4000,
                        torch.optim.Adam(model.parameters(), lr=0, betas=(0.9, 0.98), eps=1e-9))

```

Example of the curves of this model for different model sizes and for optimization hyperparameters.

```
# Three settings of the lrate hyperparameters.
opts = [NoamOpt(512, 1, 4000, None),
        NoamOpt(512, 1, 8000, None),
        NoamOpt(256, 1, 4000, None)]
plt.plot(np.arange(1, 20000), [[opt.rate(i) for opt
in opts] for i in range(1, 20000)])
plt.legend(["512:4000", "512:8000", "256:4000"])
None
```



Regularization

Label Smoothing

During training, we employed label smoothing of value $\epsilon_{ls} = 0.1$ (cite (<https://arxiv.org/abs/1512.00567>)). This hurts perplexity, as the model learns to be more unsure, but improves accuracy and BLEU

score.

We implement label smoothing using the KL div loss. Instead of using a one-hot target distribution, we create a distribution that has `confidence` of the correct word and the rest of the `smoothing` mass distributed throughout the vocabulary.

```

class LabelSmoothing(nn.Module):
    "Implement label smoothing."
    def __init__(self, size, padding_idx, smoothing=
0.0):
        super(LabelSmoothing, self).__init__()
        self.criterion = nn.KLDivLoss(size_average=F
alse)

        self.padding_idx = padding_idx
        self.confidence = 1.0 - smoothing
        self.smoothing = smoothing
        self.size = size
        self.true_dist = None

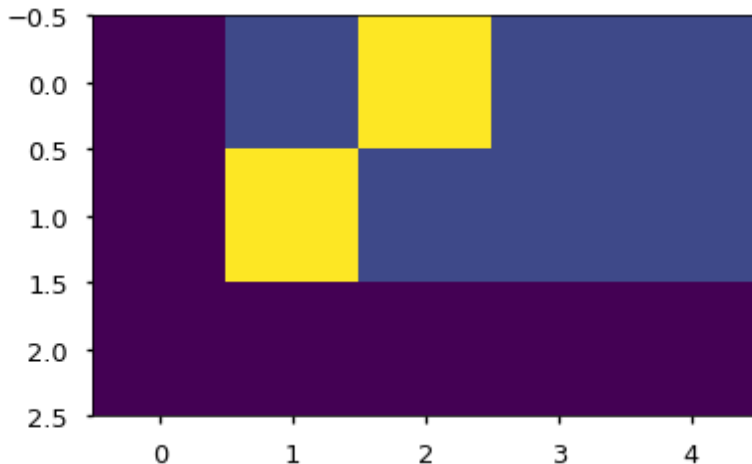
    def forward(self, x, target):
        assert x.size(1) == self.size
        true_dist = x.data.clone()
        true_dist.fill_(self.smoothing / (self.size
- 2))
        true_dist.scatter_(1, target.data.unsqueeze
(1), self.confidence)
        true_dist[:, self.padding_idx] = 0
        mask = torch.nonzero(target.data == self.pad
ding_idx)
        if mask.dim() > 0:
            true_dist.index_fill_(0, mask.squeeze(),
0.0)
        self.true_dist = true_dist
        return self.criterion(x, Variable(true_dist,
requires_grad=False))

```

Here we can see an example of how the mass is distributed to the words based on confidence.

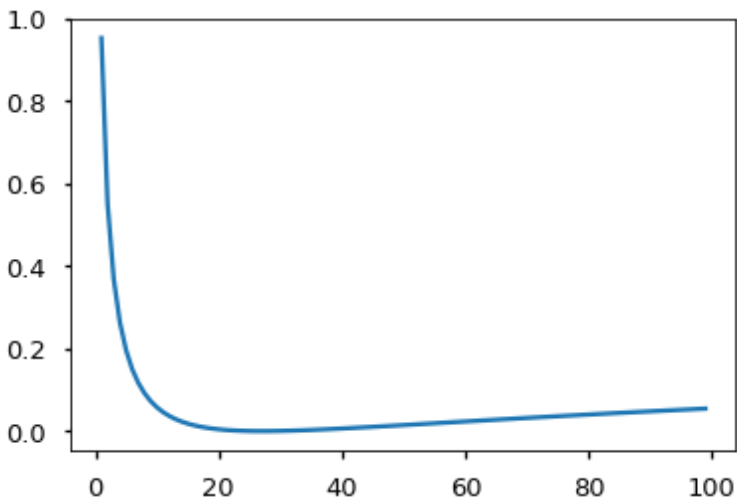
```
# Example of label smoothing.
crit = LabelSmoothing(5, 0, 0.4)
predict = torch.FloatTensor([[0, 0.2, 0.7, 0.1, 0],
                             [0, 0.2, 0.7, 0.1, 0],
                             [0, 0.2, 0.7, 0.1, 0]])
v = crit(Variable(predict.log()),
          Variable(torch.LongTensor([2, 1, 0])))

# Show the target distributions expected by the system.
plt.imshow(crit.true_dist)
None
```



Label smoothing actually starts to penalize the model if it gets very confident about a given choice.

```
crit = LabelSmoothing(5, 0, 0.1)
def loss(x):
    d = x + 3 * 1
    predict = torch.FloatTensor([[0, x / d, 1 / d, 1
/ d, 1 / d],
                                ])
    #print(predict)
    return crit(Variable(predict.log()),
                Variable(torch.LongTensor([1]))).da
ta[0]
plt.plot(np.arange(1, 100), [loss(x) for x in range
(1, 100)])
None
```



A First Example

We can begin by trying out a simple copy-task. Given a random set of input symbols from a small vocabulary, the goal is to generate back those same symbols.

Synthetic Data

```
def data_gen(V, batch, nbatches):
    "Generate random data for a src-tgt copy task."
    for i in range(nbatches):
        data = torch.from_numpy(np.random.randint(1,
V, size=(batch, 10)))
        data[:, 0] = 1
        src = Variable(data, requires_grad=False)
        tgt = Variable(data, requires_grad=False)
        yield Batch(src, tgt, 0)
```

Loss Computation

```
class SimpleLossCompute:
    "A simple loss compute and train function."
    def __init__(self, generator, criterion, opt=None):
        self.generator = generator
        self.criterion = criterion
        self.opt = opt

    def __call__(self, x, y, norm):
        x = self.generator(x)
        loss = self.criterion(x.contiguous().view(-
1, x.size(-1)),
                                y.contiguous().view(-
1)) / norm
        loss.backward()
        if self.opt is not None:
            self.opt.step()
            self.opt.optimizer.zero_grad()
        return loss.data[0] * norm
```

Greedy Decoding

```
# Train the simple copy task.
V = 11
criterion = LabelSmoothing(size=V, padding_idx=0, sm
oothing=0.0)
model = make_model(V, V, N=2)
model_opt = NoamOpt(model.src_embed[0].d_model, 1, 4
00,
                    torch.optim.Adam(model.parameters()), lr=0, b
etas=(0.9, 0.98), eps=1e-9))

for epoch in range(10):
    model.train()
    run_epoch(data_gen(V, 30, 20), model,
              SimpleLossCompute(model.generator, cri
terion, model_opt))
    model.eval()
    print(run_epoch(data_gen(V, 30, 5), model,
                    SimpleLossCompute(model.generato
r, criterion, None)))
```

Epoch Step: 1 Loss: 3.023465 Tokens per Sec: 403.074
173

Epoch Step: 1 Loss: 1.920030 Tokens per Sec: 641.689
380

1.9274832487106324

Epoch Step: 1 Loss: 1.940011 Tokens per Sec: 432.003
378

Epoch Step: 1 Loss: 1.699767 Tokens per Sec: 641.979
665

1.657595729827881

Epoch Step: 1 Loss: 1.860276 Tokens per Sec: 433.320
240

Epoch Step: 1 Loss: 1.546011 Tokens per Sec: 640.537
198

1.4888023376464843

Epoch Step: 1 Loss: 1.682198 Tokens per Sec: 432.092
305

Epoch Step: 1 Loss: 1.313169 Tokens per Sec: 639.441
857

1.3485562801361084

Epoch Step: 1 Loss: 1.278768 Tokens per Sec: 433.568
756

Epoch Step: 1 Loss: 1.062384 Tokens per Sec: 642.542
067

0.9853351473808288

Epoch Step: 1 Loss: 1.269471 Tokens per Sec: 433.388
727

Epoch Step: 1 Loss: 0.590709 Tokens per Sec: 642.862
135

0.5686767101287842

Epoch Step: 1 Loss: 0.997076 Tokens per Sec: 433.009
746

Epoch Step: 1 Loss: 0.343118 Tokens per Sec: 642.288
427

0.34273059368133546

Epoch Step: 1 Loss: 0.459483 Tokens per Sec: 434.594
030

Epoch Step: 1 Loss: 0.290385 Tokens per Sec: 642.519

464

0.2612409472465515

Epoch Step: 1 Loss: 1.031042 Tokens per Sec: 434.557
008

Epoch Step: 1 Loss: 0.437069 Tokens per Sec: 643.630
322

0.4323212027549744

Epoch Step: 1 Loss: 0.617165 Tokens per Sec: 436.652
626

Epoch Step: 1 Loss: 0.258793 Tokens per Sec: 644.372
296

0.27331129014492034

*This code predicts a translation using greedy decoding
for simplicity.*

```

def greedy_decode(model, src, src_mask, max_len, start_symbol):
    memory = model.encode(src, src_mask)
    ys = torch.ones(1, 1).fill_(start_symbol).type_as(src.data)
    for i in range(max_len-1):
        out = model.decode(memory, src_mask,
                           Variable(ys),
                           Variable(subsequent_mask
                                   (ys.size(1))
                                   .type_as(src.data)))
        prob = model.generator(out[:, -1])
        _, next_word = torch.max(prob, dim = 1)
        next_word = next_word.data[0]
        ys = torch.cat([ys,
                        torch.ones(1, 1).type_as(src.data).fill_(next_word)], dim=1)
    return ys

model.eval()
src = Variable(torch.LongTensor([[1,2,3,4,5,6,7,8,9,10]]))
src_mask = Variable(torch.ones(1, 1, 10))
print(greedy_decode(model, src, src_mask, max_len=10, start_symbol=1))

```

```

    1    2    3    4    5    6    7    8
9    10
[torch.LongTensor of size 1x10]

```

A Real World Example

Now we consider a real-world example using the IWSLT German-English Translation task. This task is much smaller than the WMT task considered in the paper, but it illustrates the whole system. We also show how to use multi-gpu processing to make it really fast.

```
#!/pip install torchtext spacy
#!/python -m spacy download en
#!/python -m spacy download de
```

Data Loading

We will load the dataset using torchtext and spacy for tokenization.

```

# For data loading.
from torchtext import data, datasets

if True:
    import spacy
    spacy_de = spacy.load('de')
    spacy_en = spacy.load('en')

    def tokenize_de(text):
        return [tok.text for tok in spacy_de.tokenizer(text)]

    def tokenize_en(text):
        return [tok.text for tok in spacy_en.tokenizer(text)]

    BOS_WORD = '<s>'
    EOS_WORD = '</s>'
    BLANK_WORD = "<blank>"
    SRC = data.Field(tokenize=tokenize_de, pad_token=BLANK_WORD)
    TGT = data.Field(tokenize=tokenize_en, init_token=BOS_WORD,
                      eos_token = EOS_WORD, pad_token=BLANK_WORD)

    MAX_LEN = 100
    train, val, test = datasets.IWSLT.splits(
        exts=('.de', '.en'), fields=(SRC, TGT),
        filter_pred=lambda x: len(vars(x)['src']) <=
MAX_LEN and
        len(vars(x)['trg']) <= MAX_LEN)
    MIN_FREQ = 2
    SRC.build_vocab(train.src, min_freq=MIN_FREQ)
    TGT.build_vocab(train.trg, min_freq=MIN_FREQ)

```

Batching matters a ton for speed. We want to have very evenly divided batches, with absolutely minimal padding. To do this we have to hack a bit around the default torchtext batching. This code patches their default batching to make sure we search over enough sentences to find tight batches.

Iterators

```
class MyIterator(data.Iterator):
    def create_batches(self):
        if self.train:
            def pool(d, random_shuffler):
                for p in data.batch(d, self.batch_size * 100):
                    p_batch = data.batch(
                        sorted(p, key=self.sort_key),
                        self.batch_size, self.batch_size_fn)
                    for b in random_shuffler(list(p_batch)):
                        yield b
            self.batches = pool(self.data(), self.random_shuffler)

        else:
            self.batches = []
            for b in data.batch(self.data(), self.batch_size,
                               self.batch_size_fn):
                self.batches.append(sorted(b, key=self.sort_key))

    def rebatch(pad_idx, batch):
        "Fix order in torchtext to match ours"
        src, trg = batch.src.transpose(0, 1), batch.trg.transpose(0, 1)
        return Batch(src, trg, pad_idx)
```

Multi-GPU Training

Finally to really target fast training, we will use multi-gpu. This code implements multi-gpu word generation. It is not specific to transformer so I won't go into too much detail. The idea is to split up word generation at training time into chunks to be processed in parallel across many different gpus. We do this using pytorch parallel primitives:

- replicate - split modules onto different gpus.
- scatter - split batches onto different gpus
- parallel_apply - apply module to batches on different gpus
- gather - pull scattered data back onto one gpu.
- nn.DataParallel - a special module wrapper that calls these all before evaluating.

```
if opt is not None)]
```

```

                                for o in out_scatter]
        gen = nn.parallel.parallel_apply(generator, out_column)

        # Compute loss.
        y = [(g.contiguous().view(-1, g.size(-1)),
                                t[:, i:i+chunk_size].contiguous().view(-1))
                for g, t in zip(gen, targets)]
        loss = nn.parallel.parallel_apply(self.criterion, y)

        # Sum and normalize loss
        l = nn.parallel.gather(loss, target_device=self.devices[0])
        l = l.sum()[0] / normalize
        total += l.data[0]

        # Backprop loss to output of transformer
        if self.opt is not None:
            l.backward()
            for j, l in enumerate(loss):
                out_grad[j].append(out_column[j][0].grad.data.clone())

        # Backprop all loss through transformer.
        if self.opt is not None:
            out_grad = [Variable(torch.cat(og, dim=1)) for og in out_grad]
            o1 = out
            o2 = nn.parallel.gather(out_grad, target_device=self.devices[0])
            o1.backward(gradient=o2)
            self.opt.step()

```

```

        self.opt.optimizer.zero_grad()
    return total * normalize

```

Now we create our model, criterion, optimizer, data iterators, and paralelization

```

# GPUs to use
devices = [0, 1, 2, 3]
if True:
    pad_idx = TGT.vocab.stoi["<blank>"]
    model = make_model(len(SRC.vocab), len(TGT.vocab), N=6)
    model.cuda()
    criterion = LabelSmoothing(size=len(TGT.vocab), padding_idx=pad_idx, smoothing=0.1)
    criterion.cuda()
    BATCH_SIZE = 12000
    train_iter = MyIterator(train, batch_size=BATCH_SIZE, device=0,
                            repeat=False, sort_key=lambda x: (len(x.src), len(x.trg)),
                            batch_size_fn=batch_size_fn, train=True)
    valid_iter = MyIterator(val, batch_size=BATCH_SIZE, device=0,
                            repeat=False, sort_key=lambda x: (len(x.src), len(x.trg)),
                            batch_size_fn=batch_size_fn, train=False)
    model_par = nn.DataParallel(model, device_ids=devices)
None

```

Now we train the model. I will play with the warmup steps a bit, but everything else uses the default parameters. On an AWS p3.8xlarge with 4 Tesla V100s, this runs at ~27,000 tokens per second with a batch size of 12,000

Training the System

```
#!/wget https://s3.amazonaws.com/opennmt-models/iwslt.pt (https://s3.amazonaws.com/opennmt-models/iwslt.pt)
```

```

if False:
    model_opt = NoamOpt(model.src_embed[0].d_model,
    1, 2000,
        torch.optim.Adam(model.parameters(), lr=
0, betas=(0.9, 0.98), eps=1e-9))
    for epoch in range(10):
        model_par.train()
        run_epoch((rebatch(pad_idx, b) for b in train_iter),
            model_par,
            MultiGPULossCompute(model.generator, criterion,
                devices=device
s, opt=model_opt))
        model_par.eval()
        loss = run_epoch((rebatch(pad_idx, b) for b
in valid_iter),
            model_par,
            MultiGPULossCompute(model.generator, criterion,
                devices=devices, opt=None))
        print(loss)
else:
    model = torch.load("iwslt.pt")

```

Once trained we can decode the model to produce a set of translations. Here we simply translate the first sentence in the validation set. This dataset is pretty small so the translations with greedy search are reasonably accurate.

```

for i, batch in enumerate(valid_iter):
    src = batch.src.transpose(0, 1)[:1]
    src_mask = (src != SRC.vocab.stoi["<blank>"]).unsqueeze(-2)
    out = greedy_decode(model, src, src_mask,
                        max_len=60, start_symbol=TG
T.vocab.stoi["<s>"])
    print("Translation:", end="\t")
    for i in range(1, out.size(1)):
        sym = TGT.vocab.itos[out[0, i]]
        if sym == "</s>": break
        print(sym, end=" ")
    print()
    print("Target:", end="\t")
    for i in range(1, batch.trg.size(0)):
        sym = TGT.vocab.itos[batch.trg.data[i, 0]]
        if sym == "</s>": break
        print(sym, end=" ")
    print()
    break

```

Translation: <unk> <unk> . In my language , that means , thank you very much .

Gold: <unk> <unk> . It means in my language , thank you very much .

Additional Components: BPE, Search, Averaging

So this mostly covers the transformer model itself. There are four aspects that we didn't cover explicitly. We also have all these additional features implemented in OpenNMT-py (<https://github.com/opennmt/opennmt-py>).

1) BPE/ Word-piece: We can use a library to first preprocess the data into subword units. See Rico Sennrich's subword- nmt (<https://github.com/rsennrich/subword-nmt>) implementation. These models will transform the training data to look like this:

`__Die __Protokoll datei __kann __ heimlich __per __E - Mail __oder
__FTP __an __einen __bestimmte n __Empfänger __gesendet
__werden .`

2) Shared Embeddings: When using BPE with shared vocabulary we can share the same weight vectors between the source / target / generator. See the (cite) (<https://arxiv.org/abs/1608.05859>) for details. To add this to the model simply do this:

```
if False:
    model.src_embed[0].lut.weight = model.tgt_embeddings[0].lut.weight
    model.generator.lut.weight = model.tgt_embed[0].lut.weight
```

3) Beam Search: This is a bit too complicated to cover here. See the OpenNMT- py (<https://github.com/OpenNMT/OpenNMT-py/blob/master/onmt/translate/Beam.py>) for a pytorch implementation.

4) Model Averaging: The paper averages the last k checkpoints to create an ensembling effect. We can do this after the fact if we have a bunch of models:

```
def average(model, models):  
    "Average models into model"  
    for ps in zip(*[m.params() for m in [model] + models]):  
        p[0].copy_(torch.sum(*ps[1:]) / len(ps[1:]))
```

Results

On the WMT 2014 English-to-German translation task, the big transformer model (Transformer (big) in Table 2) outperforms the best previously reported models (including ensembles) by more than 2.0 BLEU, establishing a new state-of-the-art BLEU score of 28.4. The configuration of this model is listed in the bottom line of Table 3. Training took 3.5 days on 8 P100 GPUs. Even our base model surpasses all previously published models and ensembles, at a fraction of the training cost of any of the competitive models.

On the WMT 2014 English-to-French translation task, our big model achieves a BLEU score of 41.0, outperforming all of the previously published single models, at less than 1/4 the training cost of the

previous state-of-the-art model. The Transformer (big) model trained for English-to-French used dropout rate $P_{\text{drop}} = 0.1$, instead of 0.3.

```
Image(filename="images/results.png")
```

Table 2: The Transformer achieves better BLEU scores than previous state-of-the-art models on the English-to-German and English-to-French newstest2014 tests at a fraction of the training cost.

Model	BLEU		Training Cost (FLOPs)	
	EN-DE	EN-FR	EN-DE	EN-FR
ByteNet [18]	23.75			
Deep-Att + PosUnk [39]		39.2		$1.0 \cdot 10^{20}$
GNMT + RL [38]	24.6	39.92	$2.3 \cdot 10^{19}$	$1.4 \cdot 10^{20}$
ConvS2S [9]	25.16	40.46	$9.6 \cdot 10^{18}$	$1.5 \cdot 10^{20}$
MoE [32]	26.03	40.56	$2.0 \cdot 10^{19}$	$1.2 \cdot 10^{20}$
Deep-Att + PosUnk Ensemble [39]		40.4		$8.0 \cdot 10^{20}$
GNMT + RL Ensemble [38]	26.30	41.16	$1.8 \cdot 10^{20}$	$1.1 \cdot 10^{21}$
ConvS2S Ensemble [9]	26.36	41.29	$7.7 \cdot 10^{19}$	$1.2 \cdot 10^{21}$
Transformer (base model)	27.3	38.1	$3.3 \cdot 10^{18}$	
Transformer (big)	28.4	41.8	$2.3 \cdot 10^{19}$	

The code we have written here is a version of the base model. There are fully trained version of this system available here (Example Models) (<http://opennmt.net/Models-py/>).

With the additional extensions in the last section, the OpenNMT-py replication gets to 26.9 on EN-DE WMT. Here I have loaded in those parameters to our reimplementenation.

```
!wget https://s3.amazonaws.com/opennmt-models/en-de-model.pt
```

```
model, SRC, TGT = torch.load("en-de-model.pt")
```

```

model.eval()
sent = "_The_log_file_can_be_sent_secretly_with_email_or_FTP_to_a_specified_receiver".split()
src = torch.LongTensor([[SRC.stoi[w] for w in sent]])
src = Variable(src)
src_mask = (src != SRC.stoi["<blank>"]).unsqueeze(-2)
out = greedy_decode(model, src, src_mask,
                    max_len=60, start_symbol=TGT.stoi["<s>"])
print("Translation:", end="\t")
trans = "<s> "
for i in range(1, out.size(1)):
    sym = TGT.itos[out[0, i]]
    if sym == "</s>": break
    trans += sym + " "
print(trans)

```

Translation: <s> _Die _Protokoll datei _kann _heimlich _per _E - Mail _oder _FTP _an _einen _bestimmten _Empfänger _gesendet _werden .

Attention Visualization

Even with a greedy decoder the translation looks pretty good. We can further visualize it to see what is happening at each layer of the attention

```

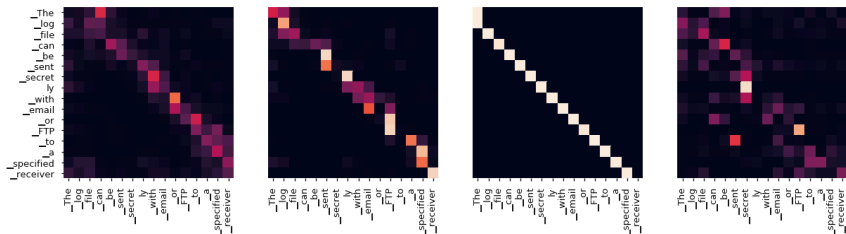
tgt_sent = trans.split()
def draw(data, x, y, ax):
    seaborn.heatmap(data,
                    xticklabels=x, square=True, ytick
klabels=y, vmin=0.0, vmax=1.0,
                    cbar=False, ax=ax)

for layer in range(1, 6, 2):
    fig, axs = plt.subplots(1,4, figsize=(20, 10))
    print("Encoder Layer", layer+1)
    for h in range(4):
        draw(model.encoder.layers[layer].self_attn.a
ttn[0, h].data,
            sent, sent if h ==0 else [], ax=axs[h])
    plt.show()

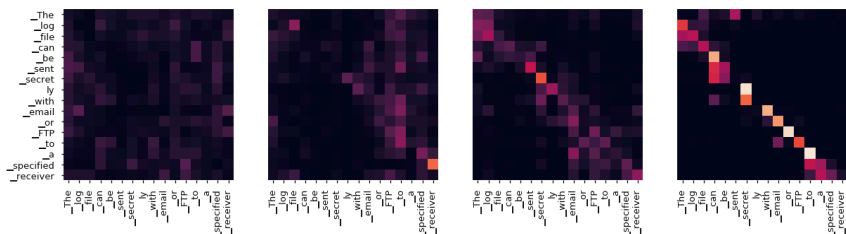
for layer in range(1, 6, 2):
    fig, axs = plt.subplots(1,4, figsize=(20, 10))
    print("Decoder Self Layer", layer+1)
    for h in range(4):
        draw(model.decoder.layers[layer].self_attn.a
ttn[0, h].data[:len(tgt_sent), :len(tgt_sent)],
            tgt_sent, tgt_sent if h ==0 else [], ax=
axs[h])
    plt.show()
    print("Decoder Src Layer", layer+1)
    fig, axs = plt.subplots(1,4, figsize=(20, 10))
    for h in range(4):
        draw(model.decoder.layers[layer].self_attn.a
ttn[0, h].data[:len(tgt_sent), :len(sent)],
            sent, tgt_sent if h ==0 else [], ax=axs
[h])
    plt.show()

```

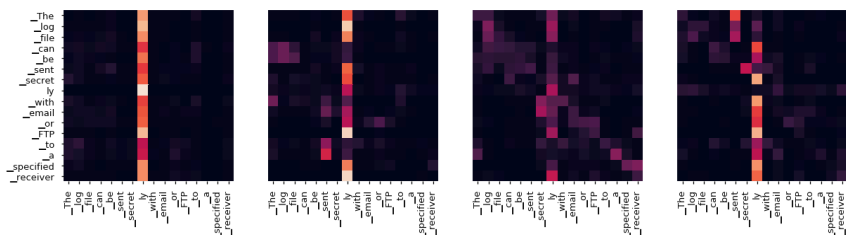
Encoder Layer 2



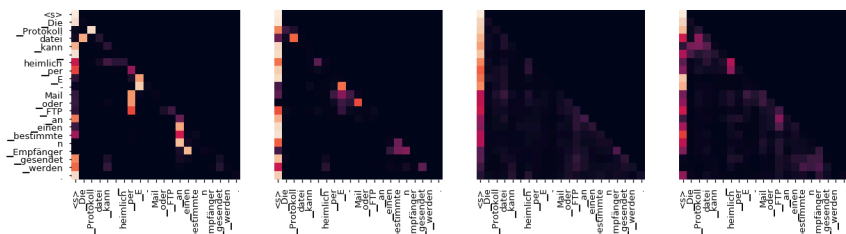
Encoder Layer 4



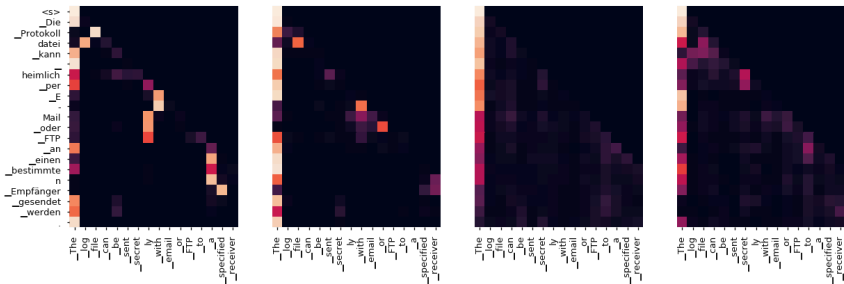
Encoder Layer 6



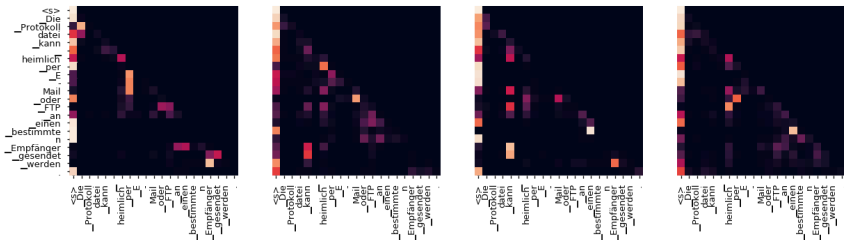
Decoder Self Layer 2



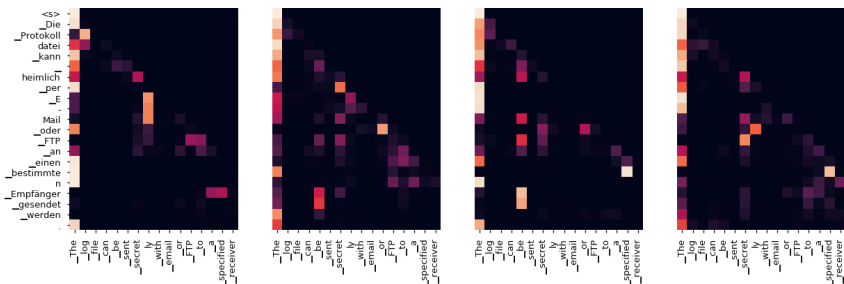
Decoder Src Layer 2



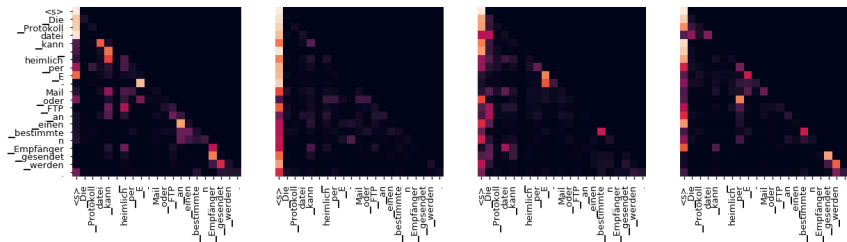
Decoder Self Layer 4



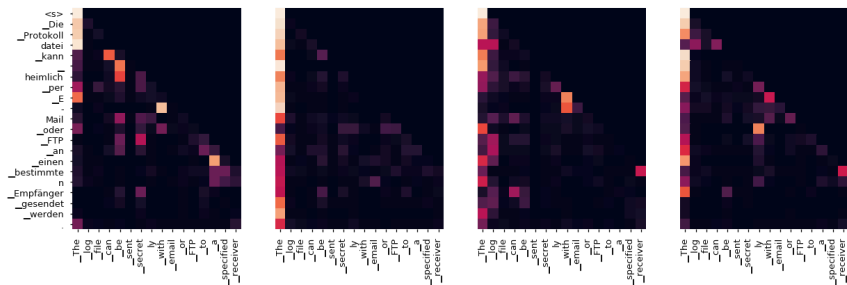
Decoder Src Layer 4



Decoder Self Layer 6



Decoder Src Layer 6



Conclusion

Hopefully this code is useful for future research. Please reach out if you have any issues. If you find this code helpful, also check out our other OpenNMT tools.

```
@inproceedings{opennmt,  
  author    = {Guillaume Klein and  
              Yoon Kim and  
              Yuntian Deng and  
              Jean Senellart and  
              Alexander M. Rush},  
  title     = {OpenNMT: Open-Source Toolkit for Neural  
Machine Translation},  
  booktitle = {Proc. ACL},  
  year      = {2017},  
  url       = {https://doi.org/10.18653/v1/P17-4012  
(https://doi.org/10.18653/v1/P17-4012)},  
  doi       = {10.18653/v1/P17-4012}  
}
```

Cheers, srush

Join the discussion...

LOG IN WITH

OR SIGN UP WITH DISQUS 

Name

 120

Share

Best Newest Oldest



free_variation

8 years ago

Thank you for a great piece.

I have a question about the positional encoding. Here you seem to interleave sine and cosine curves over the dimensions for a given position. The indexing in the original paper suggests the same, but the code at <https://github.com/tensorfl...> (and the discussion at <http://jalammargithub.io/i...>) suggests a concatenation of sines on the left and cosines on the right.

I'm not sure it makes a substantive difference, but would you agree there's a difference in implementation

would you agree there's a difference in implementation here?

10 1 Reply 

P

Paul Nguyen

 free_variation

—

6 years ago

free_variation did you ever find the answer to this?

0 0 Reply 

M

Mark Daoust

—

 Paul Nguyen

4 years ago

Yes. This is easier to implement just concatenating them instead of interleaving them. It makes no difference unless you're trying to do something like load a checkpoint from another implementation.

0 0 Reply 

G

Gus Smith

—

6 years ago edited

The loss computation appears to be updating gradients during validation.

This appears to drive weight updates to fit the validation (test) data, which it should not.

A recommended way to run validation is apparently:

```
model.eval()
```

```
torch.no_grad()
```

[<https://discuss.pytorch.org...>](url)

<https://discuss.pytorch.org...>

And indeed, the code that runs without error is the following, where `loss.backward()` is pushed under the `if` statement:

```
model.eval()
with torch.no_grad():
    test_loss = run_epoch(val_dataloader
                           SimpleLossCompute(model.generator))
    print("test_loss", test_loss)
```

[see more](#)

5 0 Reply 

P

Paul Nguyen

 Gus Smith

—

6 years ago

Hey Gus, did you end up getting this code to work? Did you want to compare results?

0 0 Reply 

K

Kaustubh Kunte

—

8 years ago

Question: Why does MultiHeadAttention class create only 4 clones instead of 8?

```
self.linears = clones(nn.Linear(d_model, d_model), 4)
```

Thanks for a great post !

7 1 Reply 



Yu-Tao

 Kaustubh Kunte

—

8 years ago

I guess you misunderstand the zip function.

```
query, key, value = \
```

```
[(x.view(nbatches, -1, self.h,
```

```
self.d_k) for x in zip(query, key, value)]
```

```
self.d_k).transpose(1, 2)
for l, x in zip(self.linears, (query, key, value))]
```

In this code, the first linear layer works on query, the second one works on key and the third one works on value. The fourth one is used in: `return self.linears[-1](x)`

So the total number of linear layers is four.

11 0 Reply 

N

nimning

→ Yu-Tao

—

8 years ago

Hi

Why the linear layer has dimension $(d_{\text{model}}, d_{\text{model}})$? The W^Q , W^K , and W^V have dimension (d_{model}, d_k) , (d_{model}, d_k) , and (d_{model}, d_v) ? I do not see any layer corresponding to any of the three matrices.

1 0 Reply 

Y

Yipu Ding

—

→ nimning

8 years ago

This is my understanding, see the code:

`query, key, value = \`

```
[l(x).view(nbatches, -1,
self.h,
self.d_k).transpose(1, 2)
for l, x in zip(self.linears,
(query, key, value))]
```

this will let the query

and will let the query,
key,value pass through the
first three Linear layers in
self.linears,
so the first three Linear
layers will be the Wq,Wk
and Wv as you mentioned.
after this the matrix of
query,key and value will

[see more](#)

4

0

[Reply](#)



F

**Francois
Steiner**

—

[→ Yipu Ding](#)

7 years ago

Thanks for a great post!
It seems to me that instead
of splitting the original
matrix in 8 for the multi-
head attention, we should
be creating 8 independent
learnt projections, so the
code should be re-written
accordingly - any
thoughts?

0

0

[Reply](#)



Sidney Melo

—

[→ Francois](#)

Steiner

7 years ago

I also believed the input would be projected entirely in 8 new vectors when I read the paper. Question is: to create 8 independent learnt projections is really equivalent to splitting the original matrix in 8?

0 0 Reply 

C

chuankang wu —

→ Sidney Melo

7 years ago

it is equivalent, an example below :



see more

9

0

Reply 

海

海尔

 Yu-Tao

—

6 years ago edited

Thanks for your answer!

0

0

Reply 

F

Frank Wang

—

7 years ago

For the `make_std_mask`` method:
`tgt_mask`` have the shape of (1, batches, length), and
the `subsequent_mask`` method should output shape
of (1, length, length). How the two tensors have ``&``
operator?

3

0

Reply 

K

Kenenbek Arzymatov

—

 Frank Wang

7 years ago

Have you found an answer?

0

0

Reply 

V

vishwaas chandan

—

 Kenenbek Arzymatov

6 years ago

I was stuck in the same question,
but then I ran the the Batch class
with a sample data, turns out the
`tgt_mask`` after unsqueeze has size:
(nbatch,1, length) and the
`subsequent_mask`` method should

output shape is (1, length, length), and it looks like the pytorch "&" operation (unlike tensorflow) automatically tiles the first tensor along 0 dimension and the second tensor along 1st dimension and gives an output of size: (nbatch, length, length), this is intuitive since we want to have masking on every row in the batch.

1 0 Reply 

P

Paul Nguyen

—

 vishwaas

chandan

6 years ago

Could you try out different inputs for the "copy and paste" task?

src =

Variable(torch.LongTensor
([[1,2,3,4,5,6,7,8,9,10]])) I

have been getting different outputs different to the inputs. Which is really strange since my

translation task works just

fine..... here my version of

the code for pytorch 1.5.1

[https://github.com/mathe
mat...](https://github.com/mathe mat...)

1 0 Reply 



gaziev

8 years ago

Thanks for great post!

Q: shouldn't it be `mask.dim() > 1` in

`LabelSmoothing#forward`?

Because in your example with target `[1]` mask after `nonzero([1] == 0)` will be `tensor([])` which has dimension 1 and it will throw error if we pass it as argument in `index_fill_` as index.

5 1 Reply



Piji Li

→ gaziev

8 years ago

`mask.sum() > 0` ?

6 1 Reply



Ke Wang

→ Piji Li

8 years ago

change it to "`mask.sum() > 0` and `len(mask) > 0`", it works!

0 1 Reply



Yu-Tao

→ gaziev

8 years ago

Same problem. I solve it by replace `MyIteator` with `data.BucketIteator`.

I'm not sure if it is related with the batch data.

0 0 Reply



Ning Ma



8 years ago

The output of each step in the decoder is fed to the bottom decoder in the next time step. So, based on this implementation, how can the decoder generate word one by one based on previous prediction? I do not see this logic in the code.

4 1 Reply



Ishaan

Ning Ma



7 years ago edited

During training, we don't need to feed the predicted word to the bottom decoder. We use teacher forcing here and feed the true word in the next time step. During testing though, we have to feed the predicted word (till <eos>) to generate the sentence. Please take a look towards the end here:

<https://towardsdatascience...>

1 0 Reply



Arthur Marques



7 years ago

In PositionalEncoding, there is a small fix discussed here: <https://stackoverflow.com/q...>

LU Jialin quote:

For me I just got the torch.arange to generate float type tensor

from

```
position = torch.arange(0, max_len).unsqueeze(1)
div_term = torch.exp(torch.arange(0, d_model, 2) * -
```

```
(math.log(10000.0) / d_model))
```

to

```
position = torch.arange(0., max_len).unsqueeze(1)
div_term = torch.exp(torch.arange(0., d_model, 2) * -
(math.log(10000.0) / d_model))
```

2 0 Reply ↗



Иван

8 years ago edited

Thanks a lot for such great post!

I guess there is a mistake in following part of code:

```
def clones(module, N):
    "Produce N identical layers."
    return nn.ModuleList([copy.deepcopy(module) for _
in range(N)])
```

"Deepcopy" copies the entire layer with it's weights what is wrong because (W_{q_1} , W_{q_2} , W_{q_3} , ..., W_{k_1} , W_{k_2} , W_{k_3} , ..., etc) must be randomly initialized. In other words "deepcopy" copies the only instance of class `nn.Linear` but N instances must be created. So I suppose that it's better to replace it by something like that:

```
def clones(params, N):
    return nn.ModuleList([nn.Linear(params) for _ in
range(N)])
```

Sincerely,

Ivan

2 0 Reply ↗

P

Peixiang Zhong

→ Иван

—

8 years ago

Later there is code for model initialization at line 20 in the notebook

0

0

Reply



Runqi Yang

—

5 years ago

The implementation of Embedding seems incorrect.
`return self.lut(x) * math.sqrt(self.d_model)`
The original paper mentioned multiplying by "`sqrt(d_model)`" because in Tensorflow implementation, the embedding weights are initialized to the inverse of `sqrt(d_model)`, so they scale it back to $N(0, 1)$. But in pytorch the initial embedding weights are already $N(0,1)$, so multiplying by `sqrt(d_model)` will make the weights unreasonably large.

1

0

Reply



Jim Everest

—

5 years ago

Thanks for this great post.

I still have some doubts about the Mask, I know it is to prevent the subsequent positions from being seen during model training. But why does the Mask mechanism only appear in Decoder's self Attention? Shouldn't the same mask mechanism be introduced in other modules of Decoder?

Before the first Multi-Headed Attention module in Decoder, there is also a residual structure, which will bring the original Output Embedding matrix(with entire

output ground truth tokens) into the subsequent “ADD & Norm” module, add this Masked matrix, and then perform following processing.

I think this will cause all GT Output tokens to be brought in in the subsequent processing, and will be passed all the way to the final output of the Decoder. Will this not lead to the leakage of the Ground Truth in the Linear and Softmax output stages?

1 0 Reply 



Gilles

5 years ago

When doing the datasets.IWSLT.splits, I have the following error : OSError: Not a gzipped file (b'

1 0 Reply 

C

Christoph Windheuser

→ Gilles

5 years ago

Same issue for me. It seems that the functionality of torchtext 0.10.0 has changed without backward compatibility and the code in this post has to be adapted to load the IWSLT database. Has someone figured out how to solve this? Please share. Thanks!

0 0 Reply 



Amdework Asefa Zemaryam

→ Gilles

5 years ago

Same issue, have you resolved it please?

Again I was trying to run with local datasets but I can't split with torch as specified as in the code.

```
(train, val, test = datasets.IWSLT.splits(
    exts=('.en', '.de'), fields=(SRC, TGT),
    filter_pred=lambda x: len(vars(x)['src']) <=
    MAX_LEN
```

```
and len(vars(x)['trg']) <= MAX_LEN))
```

How can I use this fragment for a local dataset please help.

0 0 Reply ↗

Y

yb li

→ Gilles

—

5 years ago

I have the same question as yours, do you figure it out

0 0 Reply ↗



Amdework Asefa

—

Zemaryam

→ yb li

5 years ago

not yet, as soon as you got it. please share us! thank you!

0 0 Reply ↗

B

Benjamin

—

→ Amdework

Asefa

Zemaryam

5 years ago

edited

I solved this problem, just

go to this website:
<https://wit3.fbk.eu/2016-01>, click "link" and download the "2016-01.tgz" file. In this file you can find the "2016-01/texts/de/en/de-en.tgz" file. Simply put this file in "your_work_dir/.data/iwslt/", then it works fine.

1 0 Reply 

S

Stanger
Danger

—

➔ Benjamin

5 years ago

edited

For the struggling ones in the future.

When you run the code below, it downloads the data-set in the directory mentioned in "**root**"

address.

Name any directory and it will create a folder with that name, then it proceeds to download the content "de-en.tgz" in

/myfolder/iwslt/

```
datasets.IWSLT.splits(exts=( '.de', '.en'),  
fields=(SRC, TGT),  
root="/myfolder/",
```

`filter_pred= lambda x:`

[see more](#)

1

0

[Reply](#)



Aiman Mutasem-hellh

—

harvardnlp

Copyright © 2024 The President and Fellows of Harvard College | Accessibility (<https://accessibility.harvard.edu/>) | Digital Accessibility (<https://accessibility.huit.harvard.edu/digital-accessibility-policy>) | Report Copyright Infringement (<https://www.harvard.edu/copyright-issue/>)